

Scheduling Analysis of UAV Flight Control Workloads on PREEMPT RT Linux

Replication on Raspberry Pi 4 with 4 GB RAM

TESTING AND VERIFICATION PROJECT PROPOSAL

October 2026

Abstract

This project examines the timing of a computational workload for an unmanned aerial vehicle (UAV) controller on a Raspberry Pi 4 with 4 GB RAM. The task runs at 250 Hz, giving a period of 4 ms. The experiment compares a standard Linux kernel with a kernel configured for PREEMPT_RT. The main reference is the Raspberry Pi 5 study by Giacomossi et al. [1].

The experiment measures wake-up latency and task completion under idle conditions and under additional CPU, memory and kernel activity. Moving the experiment from Pi 5 to Pi 4 tests whether the reported trends hold on a different platform. Timing values for Pi 4 will be obtained from measurements on the actual board.

No experimental data from Pi 4 are available yet. This report presents the theoretical background, block diagrams, measurement plan and expected outcomes. The appendix explains how to enable SSH on the installed operating system and begin implementation.

Contents

1 Theoretical Background.....	3
Research Problem and Scope.....	3
Real Time Requirements.....	3
Task Model and Timing Metrics.....	3
Linux Scheduling Policies.....	4
PREEMPT RT.....	5
Raspberry Pi 4 with 4 GB RAM.....	5
Limits of Experimental Conclusions.....	6
2 Block Diagrams and Experimental Procedure.....	7
Experimental Architecture.....	7
Implementation Stages.....	8
Experimental Matrix.....	9
Timing Collection.....	9
Environmental Control.....	10
Load Design for 4 GB RAM.....	10
Evaluation Criteria.....	11
Analysis and Tracing.....	11
3 Expected Outcomes.....	13
Research Hypotheses.....	13
Reference Results from the Paper.....	13
Deliverables and Assessment.....	13
4 Preliminary Status and Next Steps.....	14
Current Status.....	14
Next Steps.....	14
Conclusion.....	14
References.....	15
Appendix Getting Started with the Installed Pi System.....	16
Enable SSH and Connect.....	16
Record System Information.....	16
Test the Timer.....	16
Confirm PREEMPT RT.....	17
Retain and Transfer Results.....	17

1 Theoretical Background

Research Problem and Scope

A UAV controller must produce commands in time to respond to changes in flight state. A common architecture uses a microcontroller for flight stabilization and another computer for image processing, navigation or planning. When both types of work run on one Linux computer, the control task shares resources with other applications and background services. The reference study investigates this problem using a 250 Hz control workload [1, sections 1–3].

This project uses a Raspberry Pi 4 with 4 GB RAM as the measurement platform. The initial experiment runs control computations with input data held in memory. Sensors and motors are outside this stage of the project. This scope allows the experiment to examine operating-system effects on task timing. Sensor acquisition and actuator output require separate measurements when developing a complete controller.

The research questions are:

- How does PREEMPT_RT change wake-up latency compared with a standard kernel on the same Pi 4?
- Does load on other cores affect completion time when the main task is pinned to a dedicated CPU?
- Which tested configurations complete the workload within the 4 ms deadline?
- Can kernel traces explain the largest observed delays?

This is a replication on a different platform. The task period and scheduling parameters follow [1], while the actual hardware and software environment must be recorded. Differences may arise from the CPU, cache hierarchy, kernel version or experimental conditions.

Real Time Requirements

Correctness in a real-time system includes both the computed result and the time at which it becomes available. A late control command can leave an actuator using an older command. Its effect depends on the control law, system state and duration of the delay sequence [1, sections 2.1 and 7.4].

Requirement	Meaning of a late result	Evaluation needed
Hard real-time	A mandatory timing constraint is violated	Establish bounds and their operating conditions
Firm real-time	The result is no longer useful after its deadline	Record discarded results
Soft real-time	The result remains useful but service quality decreases	Assess delay and quality loss

The selected frequency is $f = 250$ Hz. The period is therefore $T = 1/f = 0.004$ s = 4,000 μ s. The relative deadline D equals T , following the model in [1]. Each invocation of the computational task must finish within 4 ms of its nominal release time.

Task Model and Timing Metrics

A periodic task can be represented as $\tau = (C, T, D, P)$, where C is the CPU time required by the computation, T is the period, D is the relative deadline and P is the scheduling parameter [1,

equation 1]. On a multicore Linux system, the largest measured value of C does not establish a worst-case execution time (WCET) for all conditions.

For invocation k , let r_k be the nominal release time, s_k the timestamp recorded immediately after a successful return from the sleep function, and f_k the time at which the computational payload finishes. The project uses:

$$L_k = s_k - r_k$$

$$E_k = f_k - s_k$$

$$R_k = f_k - r_k = L_k + E_k$$

L_k is the wake-up latency observed by the application. E_k is the elapsed time during the computational payload and may include time spent preempted. R_k is the response time from nominal release to completion.

Metric	Calculation or summary	Purpose
Wake-up latency L	Mean, median, p95, p99 and maximum	Assess how promptly the task starts
Payload elapsed time E	Median, p99 and maximum	Identify interruptions or greater computation cost
Response time R	p99 and maximum	Check the completion deadline
Deadline miss	$R_k > 4,000 \mu s$	Count late completions
Actual start interval	$s_k - s_{k-1}$	Assess variation in the task rhythm
Skipped releases	Number of nominal releases not executed	Identify lost cycles after overruns

The 99th percentile is a value that approximately 99% of samples do not exceed. The maximum is the largest value in the measured sample set. These metrics describe the distribution tail, including rare but large delays. Timing variation is commonly described as jitter.

The deadline check must use R . A wake-up latency below 4 ms is insufficient because the payload still needs time to execute. Timestamps are stored in nanoseconds and converted to microseconds for presentation to avoid early rounding.

Linux Scheduling Policies

Linux provides ordinary scheduling and policies intended for real-time tasks. The FIFO and RR rules are documented in sched(7) [4], while SCHED_DEADLINE is described in the kernel documentation [5]. The parameter values below follow the reference experiment [1, table 3].

Policy	Experimental parameters	Main characteristic
SCHED_OTHER	nice 0 and -19	Shares CPU time within the ordinary scheduling class
SCHED_FIFO	Priority 50 and 99	Static priority without RR-style time slicing among equal-priority threads
SCHED_RR	Priority 50 and 99	Time slicing among threads at the same priority
SCHED_DEADLINE	Budget $Q = 400$ or $800 \mu s$; $D = T = 4$ ms	Deadline scheduling with CPU budget management

The nice value changes a task's weight in the ordinary scheduling class. Nice -19 is the value chosen in the paper; -20 is the highest-priority endpoint of the nice range. Increasing the weight does not provide a deadline guarantee [4]. The ordinary scheduling algorithm also depends on the kernel version. Linux documentation describes the transition from CFS to EEVDF, so the version used in the experiment must be recorded [8].

FIFO and RR use real-time priorities from 1 to 99. With only one control thread on a dedicated CPU, the effect of RR time slicing may be small. Conclusions about each policy must follow its measured results [4].

SCHED_DEADLINE combines Earliest Deadline First (EDF) with the Constant Bandwidth Server (CBS) mechanism. Budgets of 400 and 800 μ s correspond to 10% and 20% of a 4 ms period. Timing depends on admission and the actual CPU demand. A task that exhausts its budget can be delayed until replenishment [5].

Pinning a DEADLINE task also involves root-domain constraints. An appropriate CPU domain must be established through cpuset partitioning, and the actual policy must be checked after configuration. If the kernel rejects the request, the run records the error rather than continuing under another policy [5, section 5].

PREEMPT_RT

PREEMPT_RT makes many kernel execution paths preemptible so that higher-priority tasks can run sooner. Many interrupt handlers execute as threads, and most spinlock_t locks acquire sleeping-lock semantics [2, 3]. These changes reduce the time a task may spend waiting for kernel work.

Priority inversion occurs when a high-priority task waits for a resource held by a lower-priority task. Priority inheritance raises the lock holder's priority to help it release the resource sooner. Exceptions remain for raw_spinlock_t and some interrupt paths. PREEMPT_RT therefore does not establish a timing guarantee for an entire application by itself [2, 3].

The paper reports a shorter activation sequence in the RT trace than in the standard-kernel trace [1, table 6]. This project will examine that observation on Pi 4. The actual timer path depends on the kernel version and timer type; the kernel documentation distinguishes hard and deferred timer processing [2].

Raspberry Pi 4 with 4 GB RAM

Pi 4 uses the BCM2711 processor with four Cortex-A72 cores and 1 MB of L2 cache. The Pi 5 in the paper uses Cortex-A76 cores, private L2 caches and a shared L3 cache. CPU and cache characteristics are taken from official Raspberry Pi documentation [6]; the paper supplies the Pi 5 experimental configuration [1].

Attribute	Project platform	Reference platform
Board	Raspberry Pi 4 Model B	Raspberry Pi 5
RAM	4 GB	8 GB
CPU	Four Cortex-A72 cores	Four Cortex-A76 cores
Relevant cache	1 MB L2 for the core cluster	Private L2 and shared 2 MB L3
Measured frequency	To be confirmed on the board	2.4 GHz
Software environment	Inspect the installed OS; a 64-bit	Ubuntu 24.04 with kernel 6.8

Attribute	Project platform	Reference platform
	system is planned	

Pinning the task reduces direct CPU competition, but the cores still share memory resources. Load on another core may increase cache or DRAM access time. The project tests this hypothesis by separating CPU and memory stress. An increase in payload elapsed time alone is insufficient to identify a specific memory mechanism.

Other interference sources include interrupts, background services, page faults, swap, frequency changes and temperature. On a 4 GB board, memory stress must be tested before the main experiment to avoid turning the scheduling study into a memory-exhaustion experiment.

Limits of Experimental Conclusions

A run of 10,000 task invocations at 250 Hz covers approximately 40 nominal seconds. Its maximum describes the observed samples. A statement that no deadline miss was observed must include the number of invocations and the experimental conditions.

Repeated runs and retained raw data help assess consistency. These measurements do not replace WCET analysis, blocking-time analysis or flight-safety verification. Meeting the CPU timing requirement also leaves sensor and actuator timing to be evaluated.

2 Block Diagrams and Experimental Procedure

Experimental Architecture

The Windows computer provides SSH access, source-code management and result analysis. Pi 4 runs the measurement program and stress processes. Data analysis takes place after each run to limit interference during measurement.

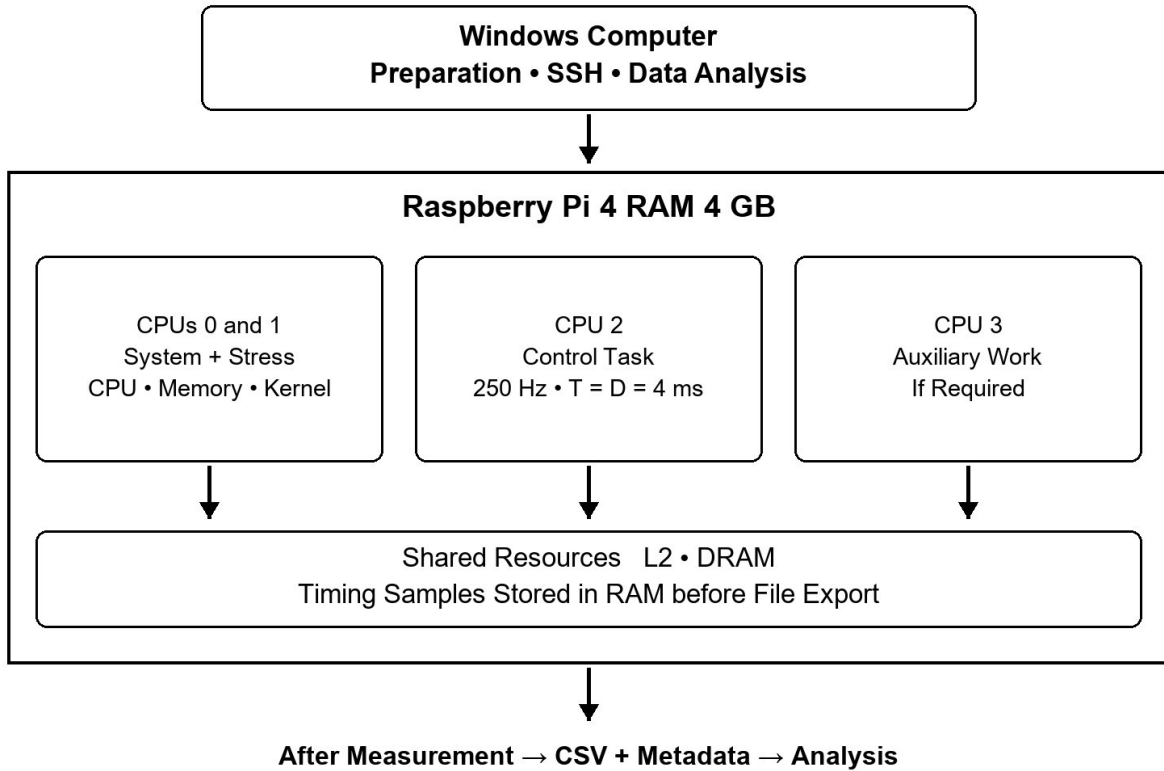


Figure 1. Proposed project architecture, following the separation of control work and background load in [1, section 4.2].

The main task runs on CPU 2. CPUs 0–1 handle the system and stress processes; CPU 3 is available for auxiliary work if needed. In each cycle, the program sleeps until the planned release, reads the clock, performs the control computation and stores the sample in RAM. Task placement must remain consistent between kernels.

The payload comprises attitude-control and output-allocation computations with predefined inputs. The authors' implementation may be used as a reference [10], with its source revision and reused components recorded. A simplified implementation must describe its function and validate its outputs before timing measurements.

Implementation Stages

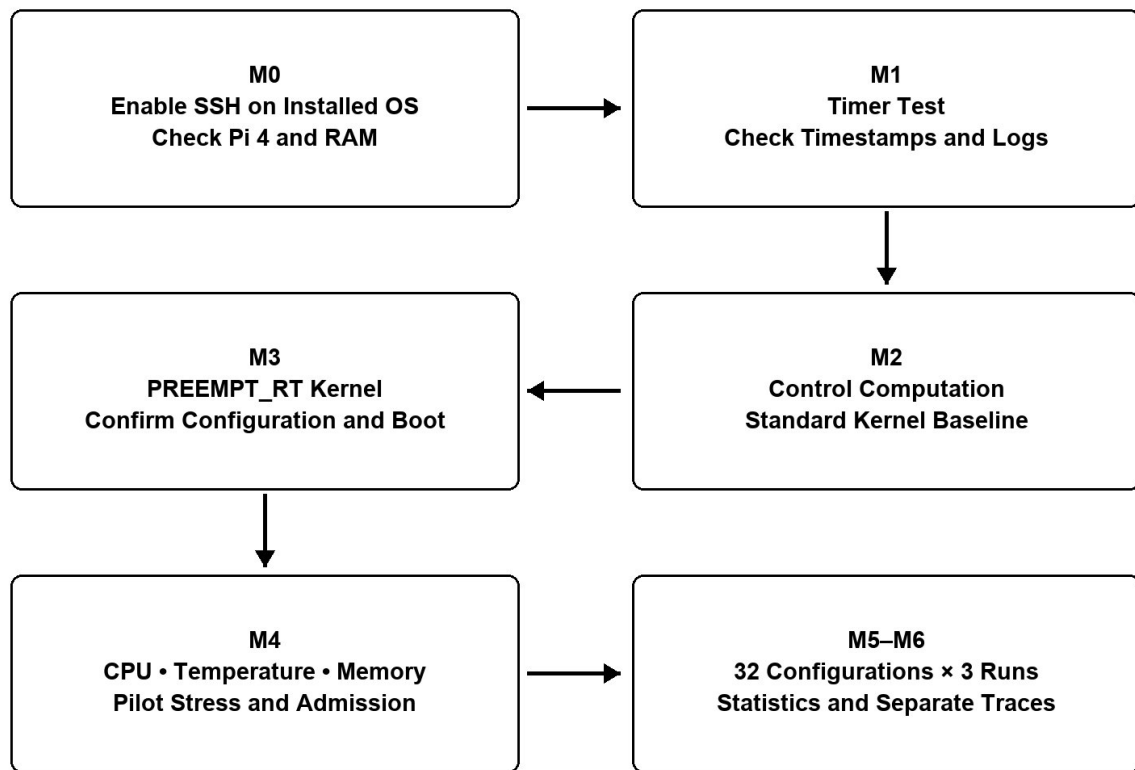


Figure 2. Planned implementation stages and the checks required before progressing.

Stage	Work	Completion condition
M0	Enable SSH and inspect the installed system	Connection works; CPU and RAM information saved
M1	Test the timer	Valid clock readings and logs
M2	Integrate the control computation on the standard kernel	Correct outputs and working CSV export
M3	Prepare a PREEMPT_RT kernel	Successful boot and confirmed RT configuration
M4	Configure CPU placement and test thermal and memory conditions	Pilot run without allocation failure or throttling
M5	Run the matrix and repetitions	Required runs and configuration records available
M6	Analyse measurements and traces	Research questions answered within the evidence

The board already has an operating system, but SSH has not been used. The immediate step is M0. The measurement program should work correctly on the standard kernel before adding RT configuration, which makes faults easier to locate.

Experimental Matrix

The matrix contains two kernels, eight scheduling configurations and two load conditions. Scheduling parameters follow [1, table 3], giving $2 \times 8 \times 2 = 32$ configurations. At least three runs per configuration are planned, for a total of 96 runs.

ID	Policy	Parameter	Period and deadline
OTHER_N0	SCHED_OTHER	nice 0	4,000 μ s
OTHER_N19	SCHED_OTHER	nice -19	4,000 μ s
FIFO_P50	SCHED_FIFO	Priority 50	4,000 μ s
FIFO_P99	SCHED_FIFO	Priority 99	4,000 μ s
RR_P50	SCHED_RR	Priority 50	4,000 μ s
RR_P99	SCHED_RR	Priority 99	4,000 μ s
DL_R400	SCHED_DEADLINE	Q = 400 μ s	D = T = 4,000 μ s
DL_R800	SCHED_DEADLINE	Q = 800 μ s	D = T = 4,000 μ s

Each run contains 500 warm-up cycles followed by 10,000 measured invocations. Warm-up and repetitions are project choices; the paper reports 10,000 invocations per test [1, section 5]. Including warm-up, the nominal workload duration of 96 runs is 67.2 minutes. Reboots, thermal stabilization and troubleshooting require additional time.

Since changing kernels requires a reboot, runs are organized in blocks. Policy and load order are randomized within each block, with the random seed saved. Waiting time and thermal conditions should be comparable between blocks.

Timing Collection

The program uses `CLOCK_MONOTONIC` for both sleep targets and timestamps. `TIMER_ABSTIME` keeps releases on the $t_0 + kT$ schedule instead of accumulating computation time into the period. If a signal interrupts the sleep and returns `EINTR`, the program retries with the same target [7].

The resolution reported by `clock_getres` does not establish wake-up accuracy. L must be measured directly. A negative L after a successful sleep return requires checking target generation, clock readings and signal handling.

The proposed sampling procedure is:

```
Set T = 4,000,000 nanoseconds
Allocate and touch memory; run warm-up cycles
Set r = current time + T
For each measured invocation:
    Sleep until r; retry the same target on EINTR
    Stop the run on any other sleep error
    Record start timestamp s
    Execute the control payload
    Record completion timestamp f
    Store r, s and f in preallocated RAM
    Set next = r + T
    Skip and count release points where next <= f
    Set r to the next release point in the future
Export CSV after measurement
```

After an overrun, the program skips elapsed release points instead of executing consecutive catch-up invocations. Both late completions and skipped releases must be reported. This overrun rule is a project implementation choice and must remain the same across configurations.

Each CSV sample records the run ID, kernel ID, policy ID, load type, invocation index, r , s , f , L , E , R and skipped-release count. Separate metadata record CPU placement, priority or budget, cycle counts, tool versions and stress commands.

Environmental Control

The two kernels should come from the same source supporting Pi 4 and use the same operating system, compiler, program revision and compiler options. If their base versions differ, this must be reported as a possible confounding factor.

Before each run, verify the thread's actual scheduling policy and CPU affinity. EPERM, EINVAL or EBUSY during policy setup makes the run invalid. Allocate and touch arrays and stack memory in advance, lock memory where permissions allow, and check the mlockall result. Collect samples in RAM and write files after the run.

Reduce background activity by running without a graphical desktop and avoiding package updates or file transfers during measurement. Keep networking and SSH available for control of the board. Record temperature, frequency and power status before, during and after runs using a consistent sampling method.

The paper sets `sched_rt_runtime_us = -1` to disable RT bandwidth throttling [1, section 5]. The initial learning stage retains the default value. Any change during the main replication must be documented and applied consistently. The control thread must still sleep periodically and have a working stop mechanism. The CBS budget of DEADLINE is evaluated separately.

Load Design for 4 GB RAM

Mixed stress combines CPU computation, memory access and kernel activity, following the reference design [1, table 2]. The tool is stress-ng [9]. Commands, tool version, process counts and allowed CPUs must be retained with the results.

Load	Placement or setup	Purpose
Idle	No stress-ng processes	Measure background interference
CPU	Compute workers on CPUs 0–1	Examine CPU load on other cores
Memory	Two workers with a defined total allocation	Examine shared memory resources
Kernel	Previously tested switch and fork stressors	Examine context switches and kernel activity
Mixed	Combine the three groups and avoid CPU 2	Main experimental matrix

For 4 GB RAM, start the pilot with approximately 1 GiB total allocation, for example two workers at 512 MiB each. This is a proposed pilot setting. Check how the installed stress-ng version interprets `--vm-bytes` and verify actual memory use before increasing the load.

Choose the allocation after the pilot and use the same byte count under both kernels. Monitor MemAvailable, out-of-memory errors and swap activity. Runs with undervoltage, thermal

throttling or allocation errors are retained with their exclusion reason and separated from the main comparison under stable conditions. Large scheduling delays under valid conditions remain part of the data.

The main matrix compares idle and mixed stress. Separate load types are additional diagnostic tests on FIFO_P50 and FIFO_P99 under both kernels. If stress is also placed on CPU 2, label it as a separate experiment because it changes the dedicated-core condition.

Evaluation Criteria

The project checks computational outputs, completion timing and actual configuration. The following criteria are defined for this project.

ID	Condition	Permitted conclusion
TC01	Outputs match the validation dataset	Payload correct for the tested inputs
TC02	Period, sleep handling and timestamps correct	Measurement program follows the design
TC03	Actual policy and CPU match configuration	Run uses the intended configuration
TC04	Every $R_k \leq 4 \text{ ms}$; no skipped releases	No deadline violation observed in the run
TC05	$\max(L) \leq 500 \text{ } \mu\text{s}$	Project wake-up target met
TC06	No memory error, undervoltage or throttling	Suitable for the main comparison
TC07	Complete CSV and valid timing relationships	Data integrity checks passed

The 500 μs threshold in TC05 is an exploratory project target, corresponding to 12.5% of the period. It is not evidence of flight stability. TC04 and TC05 are reported separately, and thresholds remain fixed after results become available.

E may include preemption time, so it cannot directly determine a DEADLINE CPU budget. The pilot must monitor budget exhaustion or add thread CPU-time measurements using `CLOCK_THREAD_CPUTIME_ID`. Budgets other than 400 and 800 μs are reported as extensions.

Analysis and Tracing

Summarize L per run using mean, median, p95, p99 and maximum. For E and R, report median, p99 and maximum, together with deadline misses and skipped releases. Present individual repetitions and their observed range rather than hiding run-to-run differences by pooling all samples.

Use time series to identify delay bursts and box plots or empirical cumulative distributions to compare configurations. On R plots, mark the 4,000 μs deadline. On L plots, mark the 500 μs wake-up target. Every plot must state its units and load condition.

For a metric M, the reduction is $\Delta = 100 \times (M_{\text{standard}} - M_{\text{RT}})/M_{\text{standard}}$. Specify whether M is a percentile or a maximum. Compare runs under matching conditions rather than selecting the best run from one group and the worst from another.

Collect kernel traces in separate diagnostic runs because observation tools can add interference. Candidate events include `sched_wakeup`, `sched_switch`, `hrtimer` and `IRQ/softirq` activity. Associate events by thread ID, CPU and timer identity where the trace provides sufficient information. Check clock sources before aligning trace timestamps with application samples. A trace summary records time, CPU, thread, event type and the task being awakened.

To investigate memory effects, compare CPU stress with memory stress. If supported by the hardware, collect cache or stalled-cycle events in separate runs. Timing data alone support a statement that results are consistent with memory interference, rather than proving a specific cache mechanism.

Reported limitations include kernel-version differences, temperature, run order, background load and measurement overhead. The CPU experiment does not cover all I/O drivers or the complete flight-control system.

3 Expected Outcomes

Research Hypotheses

The first hypothesis is that PREEMPT_RT reduces large delays with FIFO, RR or DEADLINE under mixed stress. Evaluation uses p99, maximum latency and repeated runs.

The second hypothesis is that memory load on another core still increases E or R when the main task is pinned. Separate load tests will help identify the size of this effect.

The third hypothesis is that some RT configurations meet TC04 and TC05 on Pi 4. Failures require examining configuration, payload behaviour and traces. These are proposed hypotheses.

Reference Results from the Paper

The table cites maximum latencies under stress from [1, table 4, page 5], measured on Pi 5 with 8 GB RAM. Units are microseconds; commas separate thousands.

Policy	Standard kernel on Pi 5	PREEMPT_RT on Pi 5
OTHER nice 0	8,626	9,015
OTHER nice -19	9,424	4,351
FIFO priority 50	700	160
FIFO priority 99	1,848	224
RR priority 50	787	225
RR priority 99	472	182
DEADLINE Q = 400 μ s	345	209
DEADLINE Q = 800 μ s	443	197

For FIFO priority 99, the reduction calculated from the table is $(1,848 - 224)/1,848 \times 100 \approx 87.9\%$. The ratio $224/4,000 \times 100 = 5.6\%$ expresses maximum wake-up latency relative to the period. These calculations use Pi 5 data from [1] and do not define a required Pi 4 result.

SCHED_OTHER nice 0 has a larger maximum under RT in this table. FIFO priority 50 also has a smaller maximum than priority 99 under RT. Each configuration therefore needs its own measured evaluation. Changing the kernel or increasing priority does not guarantee improvement in every metric. The maximum of 225 μ s applies to the reported RT policies under these conditions, not to SCHED_OTHER.

Deliverables and Assessment

The final experiment will produce a measurement program, raw CSV files, configuration metadata, statistical summaries, distribution plots and representative traces. The complete matrix contains 32 configurations with at least three valid runs each. Any missing configurations or runs must be listed.

A configuration meeting both TC04 and TC05 is a candidate for further integration. If it fails, report the number of misses, delay sequences and conditions. Change one factor at a time in follow-up tests so that its effect can be interpreted.

4 Preliminary Status and Next Steps

Current Status

No experimental logs or timing data have been collected on Pi 4. The report therefore cannot yet give Pi 4 percentiles, maxima or configurations that meet the deadline. The numerical results in section 3 are from the Pi 5 experiment in [1].

The Pi 4 with 4 GB RAM already has an operating system, but SSH has not been used. The test plan and sample definitions are established. The installed OS, kernel, measurement program and actual environment still need to be checked.

Item	Status	Evidence required
Device	Pi 4 with 4 GB RAM selected	Board, CPU and RAM information
Operating system and SSH	OS installed; SSH not yet used	Version and successful connection
Test design	Presented in this report	Executable program and configuration
PREEMPT_RT	Not yet confirmed	Configuration of the running kernel
Pi 4 timing data	Not yet available	L, E, R, misses and skipped releases

Next Steps

First, enable SSH, identify the account and IP address, and connect from Windows. Save system information before changing the environment. Consider Raspberry Pi OS Lite 64-bit only if the installed system is unsuitable for the planned experiment.

Next, check the compiler and run a timer test on the standard kernel. Once valid logs are available, integrate the control payload and validate its output. Then prepare the RT kernel, confirm its configuration, test the stress setup and run the main matrix.

The first data milestone is a valid CSV from SCHED_OTHER nice 0 on the standard kernel under idle conditions, accompanied by board information and valid timestamps. It provides the starting point for comparing other configurations.

Conclusion

This project evaluates a 250 Hz control workload on Pi 4 with 4 GB RAM by comparing Linux kernels and scheduling policies. The report currently establishes the experimental design. Quantitative conclusions will be added after measurement and stated with their operating conditions and limits when moving to a system with real sensors and actuators.

References

- [1] Luiz Giacomossi, Håkan Forsberg, Baran Çürüklü, Ivan Tomasic and Tommaso Cucinotta. Scheduling Analysis of UAV Flight Control Workloads on PREEMPT_RT Linux Using a Raspberry Pi 5. arXiv:2604.19275v2, 2 June 2026. <https://arxiv.org/abs/2604.19275v2>
- [2] The Linux Kernel Documentation. How realtime kernels differ. <https://docs.kernel.org/core-api/real-time/differences.html>
- [3] The Linux Kernel Documentation. Lock types and their rules. <https://docs.kernel.org/locking/locktypes.html>
- [4] Linux man-pages project. sched(7). <https://man7.org/linux/man-pages/man7/sched.7.html>
- [5] The Linux Kernel Documentation. Deadline Task Scheduling. <https://docs.kernel.org/scheduler/sched-deadline.html>
- [6] Raspberry Pi Ltd. Processors, sections on BCM2711 and BCM2712. <https://www.raspberrypi.com/documentation/computers/processors.html>
- [7] Linux man-pages project. clock_nanosleep(2). https://man7.org/linux/man-pages/man2/clock_nanosleep.2.html
- [8] The Linux Kernel Documentation. CFS Scheduler. <https://docs.kernel.org/scheduler/sched-design-CFS.html>
- [9] Colin Ian King and contributors. stress-ng, official source repository. <https://github.com/ColinIanKing/stress-ng>
- [10] Luiz Giacomossi. Real-Time-Flight-Control-With-Linux, research implementation repository. <https://github.com/luizgiacomossi/Real-Time-Flight-Control-With-Linux>
- [11] Raspberry Pi Ltd. Getting started. <https://www.raspberrypi.com/documentation/computers/getting-started.html>
- [12] Raspberry Pi Ltd. The Linux kernel. https://www.raspberrypi.com/documentation/computers/linux_kernel.html
- [13] Raspberry Pi Ltd. Remote access, SSH section. <https://www.raspberrypi.com/documentation/computers/remote-access.html>
- [14] rt-tests and Debian manpages. cyclicttest(8). <https://manpages.debian.org/testing/rt-tests/cyclicttest.8.en.html>

Online sources were checked for the project on 7 October 2026. During implementation, retain the actual tool versions and source revisions. Check cyclicttest options against the help output of the version installed on Pi.

Appendix Getting Started with the Installed Pi System

Enable SSH and Connect

SSH allows commands to be entered on Pi from Windows over the network. If a monitor and keyboard are available, open a terminal on Pi and check the operating system, account and IP address:

```
cat /etc/os-release
whoami
hostname -I
```

On Raspberry Pi OS, run `sudo raspi-config`, select Interface Options, SSH, then Enable/Yes [13]. Check the service with `systemctl status ssh`. For another Linux distribution, identify the OS before choosing the method to install or enable the OpenSSH server.

Keep Pi and Windows on the same network. Open PowerShell on Windows and replace `your_user` with the Pi account and `pi_IP` with its IP address:

```
ssh your_user@pi_IP
```

Verify the device before accepting its host key on the first connection. Password entry may display no characters. If no monitor is available and SSH is disabled, determine the available access method before modifying the card; retain the installed environment.

Record System Information

After connecting, enter the following commands in the Pi SSH session:

```
mkdir -p ~/uav-rt/{src,logs,metadata}
cd ~/uav-rt
uname -a > metadata/kernel.txt
cat /etc/os-release > metadata/os.txt
lscpu > metadata/cpu.txt
free -h > metadata/memory.txt
cat /proc/device-tree/model
vcgencmd measure_temp
vcgencmd get_throttled
```

Check for a 64-bit architecture, four CPUs and the expected RAM capacity. Reported usable RAM may be below exactly 4 GB. If `vcgencmd` is unavailable, read temperature through `sysfs` and record the alternative method. `get_throttled` includes current status and historical flags; save values at both the start and end of a run.

Test the Timer

On Raspberry Pi OS or another suitable Debian-based system, install tools during preparation and retain consistent versions during measurement:

```
sudo apt update
sudo apt install build-essential git stress-ng rt-tests
gcc --version
stress-ng --version
cyclicttest --help
```

Use cyclicttest for one thread on CPU 2, priority 50, a 4,000 μ s interval and 2,500 iterations [14]:
`sudo cyclicttest -m -t1 -a2 -p50 -i4000 -l2500 -q`

This command tests timer latency without the UAV payload. Check the installed options through `cyclicttest --help` and save the output. Resolve permission or memory-locking errors before longer tests.

Once the idle test works, try CPU stress in a second SSH session:

```
taskset -c 0,1 stress-ng --cpu 2 --timeout 15s --metrics-brief
```

This is a CPU-only pilot. Memory stress is added after the environment is checked. The main measurement program from section 2 still needs to be implemented, compiled and validated before its output can be reported as experimental results.

Confirm PREEMPT RT

Selecting FIFO with `chrt` or `cyclicttest` does not turn a standard kernel into `PREEMPT_RT`. Inspect the running kernel and configuration where the system provides them:

```
uname -r
test -r /sys/kernel/realtime && cat /sys/kernel/realtime
test -r /proc/config.gz && zgrep CONFIG_PREEMPT_RT /proc/config.gz
test -r /boot/config-$(uname -r) && grep CONFIG_PREEMPT_RT /boot/config-$(uname -r)
```

`CONFIG_PREEMPT_RT=y` is important configuration evidence. If these files are absent, obtain the configuration from the corresponding package or build.

Use an RT package confirmed to support Pi 4, or build from source supporting the board. For a custom build, follow Raspberry Pi documentation [12] and retain the branch, source revision and `.config` file. Prefer standard and RT kernels from the same source, and retain the baseline environment for recovery. Exact installation commands depend on the installed OS and boot setup.

After confirming RT, repeat the timer test, check CPU affinity and pilot FIFO at priority 50. DEADLINE and higher priorities follow once configuration checks and a stop mechanism work.

Retain and Transfer Results

Keep system information, temperature and power status, `cyclicttest` results and any errors. Once the main program works, retain its CSV, compiler options and source revision. Transfer data after a run using PowerShell on Windows:

```
scp -r your_user@pi_IP:~/uav-rt/logs .
scp -r your_user@pi_IP:~/uav-rt/metadata .
```

The first milestone is a working SSH connection and a completed timer test. The next is a valid CSV from the control computation on the standard kernel.